# Centralized Student Portal and Automated Clearance System for Pamantasan ng Cabuyao

Justinbril Abarquez
*Project Manager*

Aaron Cruzante
*Back-End Developer*

Ron Canayong
*Back-End Developer*

Justine Esgana
*Front-End Developer*

Arvien Espedilon
*Front-End Developer*

Byrone Dominado
*Front-End Developer*

*Bachelor of Science in Computer Science, Pamantasan ng Cabuyao (Cabuyao University), Cabuyao, Laguna, Philippines*

***Abstract*—Students at Pamantasan ng Cabuyao queue in person to enroll, collect clearance signatures from several offices, and request records. This paper presents a web-based Centralized Student Portal and Automated Clearance System that moves these processes online for five roles: Student, Registrar, Cashier, Department head and Admin, plus a public application site for new students. The system uses a Backend-as-a-Service architecture (Next.js on Vercel and Supabase PostgreSQL with Authentication) and a database normalized to the Third Normal Form with 29 tables. Business rules, such as a department cannot clear a student with an active hold and the Registrar can only approve a clearance when every office has cleared, are enforced by database triggers and Row Level Security instead of application code. Further features are a virtual queue with a rotating QR pass, in-app notifications, and an append-only audit log. The project followed Agile sprints, was modeled with Use Case, Activity, Data Flow and Class diagrams, and was verified with 128 unit tests, about 250 database test cases, browser end-to-end checks, performance measurements and security tests. All main pages load in under 2.5 s and database queries run in under 2 ms on the test data. We also analyze the ethical and legal duties of handling student records under the Data Privacy Act of 2012.**

***Keywords*—student portal, clearance automation, row level security, Supabase, database triggers, role-based access control, Agile, data privacy**

## I. INTRODUCTION

Enrollment, clearance and document requests are routine services of a university, but in many schools they still depend on paper forms and physical lines. A student who needs clearance must visit the Cashier, each academic department and the Registrar, and every office keeps its own record. This causes long waiting times, lost or inconsistent records, and no single place where a student can see what is holding a clearance back.

The goal of this project is a single portal that (1) removes physical queuing for enrollment and document claiming, (2) automates the clearance chain across academic and financial offices, (3) gives students real-time access to their grades and status, and (4) gives staff tools that enforce the school's rules the same way every time. The contributions of this paper are the system design, the decision to enforce rules inside the database, and the evidence from testing that the system meets its functional, security and performance requirements.

## II. BACKGROUND AND RELATED WORK

**Student information systems.** Student portals usually combine records, enrollment and payments in one application. We scoped ours to six modules (Student, Registrar, Cashier, Department, Admin, Document Requests) and later added admissions, a virtual queue and notifications, because these were the main pain points reported for the Registrar window.

**Backend-as-a-Service.** To deliver in 12 days without losing architectural quality, we used Supabase, which provides PostgreSQL, authentication and an API [3]. Access control uses PostgreSQL Row Level Security (RLS) policies [4], so the database, not the web server, decides which rows a user may read or write.

**Normalization.** The schema follows the relational model [5] and is normalized to the Third Normal Form (3NF) to avoid duplicated and inconsistent data.

**Process and requirements.** We followed an Agile, Scrum-style process [6] and wrote requirements in the style of ISO/IEC/IEEE 29148 [7] with traceable IDs (REQ, SEC, PER, ETH).

## III. METHODOLOGY

**A. Process.** The team worked in four short sprints: modeling and database design; authentication and role routing; the clearance chain on live data; and enrollment, grades, documents, queue, notifications, admissions, hardening and release. Work was done on feature branches, merged by pull request into a *dev* branch, and released to *main* by the Project Manager, which Vercel deploys automatically.

**B. Requirements.** Requirements were grouped by module: Student (6), Registrar (4), Cashier (3), Department (3), Admin (4), Document requests (2), Admissions (3), Virtual queue (4), Notifications (1), plus security (8) and performance (3) requirements.

**C. Modeling.** The system was modeled with a Use Case diagram (6 actors), an Activity diagram of the term clearance (5 swimlanes), a level-1 Data Flow diagram (13 processes, 14 data stores), a Class diagram (28 classes), and an Entity Relationship diagram (29 tables). The diagrams are available at https://www.figma.com/design/caIH5zcYI3ursQ9c2iYFJg and the wireframes of 30 screens at https://www.figma.com/design/ZtlUFDtPCuTFA1V2SCIlrb.

TABLE I. SYSTEM REQUIREMENTS BY MODULE

| Module | Main requirements |
|---|---|
| Student | Sign in with student ID; see enrollment status, grades and clearance; submit enrollment; request documents and clearance |
| Registrar | Encode grades; process documents; approve or reject enrollments; final clearance only when every item is cleared |
| Cashier | View balances; log payments; tag charges; Finance clearance updates automatically |
| Department | See only own students; place and lift holds; approve departmental clearance |
| Admin | Manage accounts and roles; maintain the academic catalog; read the audit log |
| Admissions | Public application form; accept or reject; account and first-year enrollment created automatically |
| Queue, notifications | Rotating QR pass, ticket, hold and cancel; in-app inbox for every role |
| Security, performance | Authentication, RLS, lockout, rate limiting, injection protection, secret handling, password rules, headers; pages under 2.5 s; queries under 500 ms |

**D. What the diagrams show.** The Use Case diagram has six actors (Applicant, Student, Registrar, Cashier, Department head, Admin); all signed-in actors sign in first, and some use cases include or extend others (for example, issuing the final clearance includes verifying that every office has cleared). The Activity diagram follows one term clearance across five swimlanes (Student, Department, Cashier, Registrar, System) and marks the steps that the database performs by itself. The Data Flow diagram is split into ten sections (authentication, admissions, enrollment, grades, payments, clearance, document requests, queue, notifications, administration and audit); each section shows the entities, one or more processes and the data stores they touch, and dashed flows mark changes made by triggers. The Class diagram shows an abstract *User* specialized by the five role classes, each with its operations, and the domain classes (for example *Enrollment*, *Clearance*, *QueueTicket*) with their multiplicities.

## IV. SYSTEM DESIGN

**A. Architecture.** Fig. 1 shows the architecture. The browser talks to a Next.js 16 application on Vercel. A proxy refreshes the session and routes each role to its own module. Server Components and Server Actions query Supabase with the signed-in user's own session, so RLS applies to every request. A privileged server-side key is used only in server code for a few tasks, such as creating a student account when an application is accepted.

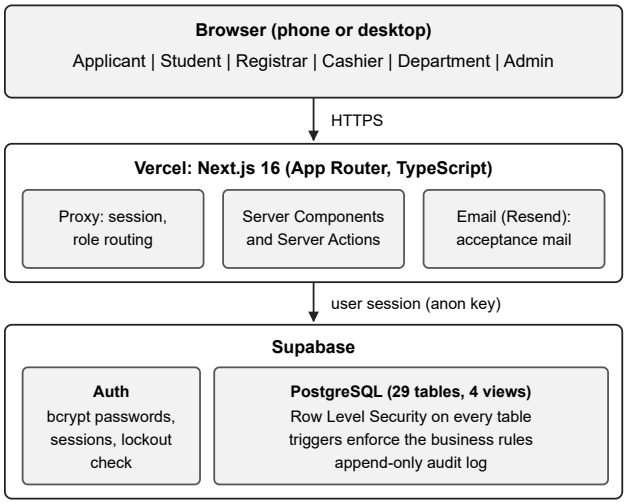



Fig. 1. System architecture.

**B. Database design.** The database has 29 tables in ten groups (identity, academic structure, enrollment and grades, admissions, finance, document requests, clearance, queue, notifications, security and audit), 47 foreign keys and four views. Design decisions that keep it in 3NF include: credentials stay in the authentication service and portal tables reference the same identifier; a grade stores only its code, with the numeric value and pass flag in a scale table; a balance is computed from charges and payments instead of stored; and the status of a clearance, an item or a hold is derived from timestamps in views. Foreign keys use *ON DELETE RESTRICT*, so accounts are deactivated, never deleted.

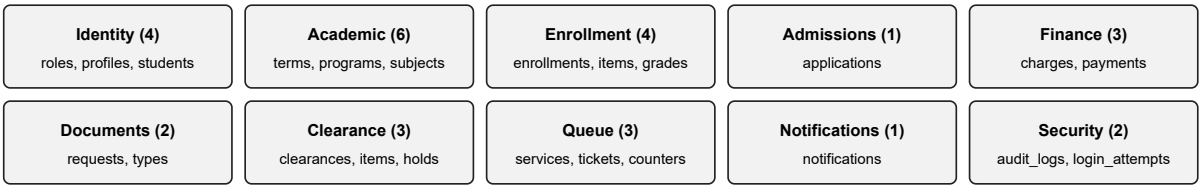



Fig. 2. The 29 tables grouped in ten modules (number of tables in brackets).

**C. Rules in the database.** Rules that must hold for every role are triggers: a department cannot clear its item while the student has an active hold of that department; the Finance item clears automatically when every charge of the student for the term is Cleared or Waived; a clearance can be approved only when all its items are cleared; grades can be posted only for an Approved enrollment; and document requests and queue tickets move only along fixed status flows. Fig. 3 shows the clearance flow that these rules control.

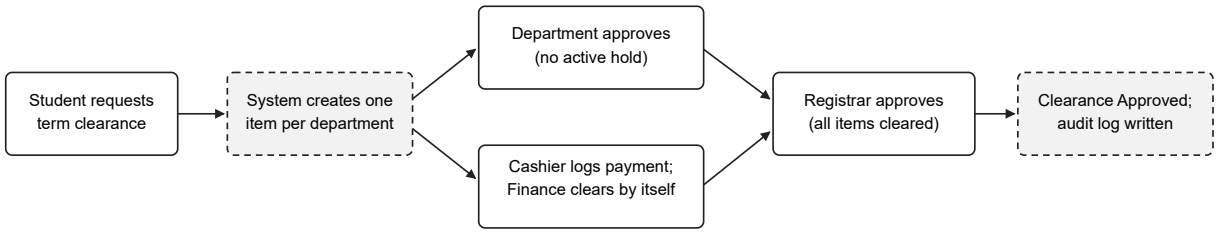



Fig. 3. Term clearance flow. Dashed boxes are automatic (database).

**D. Security design.** Passwords are hashed by the authentication service. RLS is enabled on all 29 tables and each role receives only the column-level privileges its screens need. The audit log is written by triggers and is append-only for every role, including the Admin. Sign-in has a lockout with exponential backoff, a rate limit of 20 POST requests per minute per IP, password rules (8 to 72 characters with lowercase, capital, digit and symbol), and HTTP security headers.

**E. UI/UX.** All signed-in roles share one layout (sidebar, top bar, content cards and tables) built from the same components and design tokens, with light and dark themes. Status is shown in words, disabled buttons explain why, and every page works at phone width.

## V. IMPLEMENTATION

The prototype is deployed at https://cabuyaouni.vercel.app. **Student:** dashboard with live grades, GWA per term and clearance chain; enrollment; grades; clearance; document requests (one to five documents per request, each with its own ticket); queue. **Registrar:** applications, enrollments, grade encoding, document processing, final clearance and the queue desk. **Cashier:** charges, payments and Finance clearance. **Department:** clearance items and academic holds. **Admin:** user accounts, academic setup (terms, programs, departments, subjects, sections) and the audit log.

**Admissions.** An applicant fills a public form. When the Registrar accepts it, the system creates the account with the next student number, e-mails the student ID and a link to set a password, and, for a new student, creates a Pending first-year enrollment from the program curriculum.

**Virtual queue.** A student's queue pass is a QR code carrying a signed token (HMAC-SHA256 with a server secret) that changes every 15 seconds, so a screenshot is useless after at most 30 seconds. The Registrar scans it, which issues a ticket with a random 7-digit code; the line order is kept separately so the line cannot be guessed. A student can see the place in line, hold the place for 30 minutes, or cancel. Expired holds are closed by the database.

**Notifications.** Database triggers write in-app notifications for queue, enrollment, document, clearance, hold, grade and application events. A failing notification can never block the original action.

## VI. TESTING AND RESULTS

Testing used six levels (Table II). Each database change was first rehearsed in a rolled-back transaction, then run live, then the regression script was re-run.

TABLE II. TEST LEVELS AND EVIDENCE

| Level | Method | Evidence |
|---|---|---|
| Unit | Vitest on pure functions | 128 tests pass |
| Integration | SQL suites on the live database | TC-01 to TC-259 |
| System | Browser walks per role, phone width, dark mode | E2E-01 to E2E-13, S-01 to S-12 |
| Acceptance | One check per requirement | ACC-01 to ACC-10 |
| Performance | Timed queries and page loads | Table III |
| Security | Isolation, injection, lockout, flooding | Table IV |

**Performance.** Queries for grades and clearance ran 30 times as a student with RLS on. Main pages of all five roles were timed on the live site (Table III).

TABLE III. PERFORMANCE RESULTS

| Requirement | Result | Limit |
|---|---|---|
| PER-02 query time | 0.09 to 1.46 ms | 500 ms |
| PER-01 full page load | 0.93 to 2.18 s | 2.5 s |

**Security.** Seven vulnerabilities were addressed with a solution and a test (Table IV). In addition, 24 of 24 local repository checks passed, production dependencies had no known vulnerabilities, the seven expected security headers were present on the live site, the anonymous API was refused on 17 of 17 probes, and reads between two student accounts were refused on 9 of 9 probes.

TABLE IV. VULNERABILITIES AND SOLUTIONS

| Vulnerability | Solution |
|---|---|
| Helper function callable by anonymous role | EXECUTE revoked; test TC-81 |
| Brute-force sign-in | Lockout with exponential backoff |
| Request flooding | Rate limit, HTTP 429 (20 of 40 blocked) |
| Document rule conflict | Database sets the fields; 14 cases |
| SQL injection | Parameterized calls; payloads stored as text |
| QR forgery and replay | Signed token, 15 s window |
| Proxy queuing | Random codes; ID checked at the window |

## VII. ETHICAL AND LEGAL CONSIDERATIONS

The system holds grades, balances, holds and requests of real students, so a leak or a wrong record can cause harm. We applied the ACM and IEEE codes of ethics [1], [2] by theme: avoid harm, protect privacy, be honest, and be accountable. Under the Data Privacy Act of 2012 [8], the system limits data to enrollment, grades, documents and clearance (legitimate purpose, proportionality), lets each user read only their own rows and each department only its own items (confidentiality), enforces rules for every role (integrity), and records every critical change in an append-only log (accountability). All test data are fictional.

Proposed safeguards for real use are a privacy notice for students, a written incident procedure that names who notifies the National Privacy Commission, read logging for sensitive tables, a second approver for amount changes by the Cashier, automatic retention of old logs, and a review by the university's data protection contact before real data is loaded. An AI coding assistant helped draft code and documents; the team reviewed and tested every output and approved every change to the live database.

## VIII. CONCLUSION AND FUTURE WORK

The Centralized Student Portal replaces physical queues and paper clearance with one role-based web system whose critical rules are enforced by the database. Evidence from unit, integration, system, performance and security tests shows that the main requirements are met. Future work includes a Realtime queue display, a student photo on the queue card, larger-scale load tests on a phone and slower networks, payment gateway integration, and the safeguards above.

## REFERENCES


1. ACM, “ACM Code of Ethics and Professional Conduct,” Association for Computing Machinery, 2018.
2. IEEE, “IEEE Code of Ethics,” IEEE Policies, Section 7.8.
3. Supabase Inc., “Supabase Documentation: Database and Auth,” 2026. [Online]. Available: https://supabase.com/docs
4. PostgreSQL Global Development Group, “PostgreSQL Documentation: Row Security Policies,” 2026. [Online]. Available: https://www.postgresql.org/docs/
5. E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.
6. K. Schwaber and J. Sutherland, “The Scrum Guide,” Nov. 2020.
7. ISO/IEC/IEEE, “Systems and software engineering: Life cycle processes: Requirements engineering,” ISO/IEC/IEEE 29148:2018, 2018.
8. Republic of the Philippines, “Data Privacy Act of 2012,” Republic Act No. 10173, Aug. 2012.
9. Vercel Inc., “Next.js Documentation,” 2026. [Online]. Available: https://nextjs.org/docs